



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



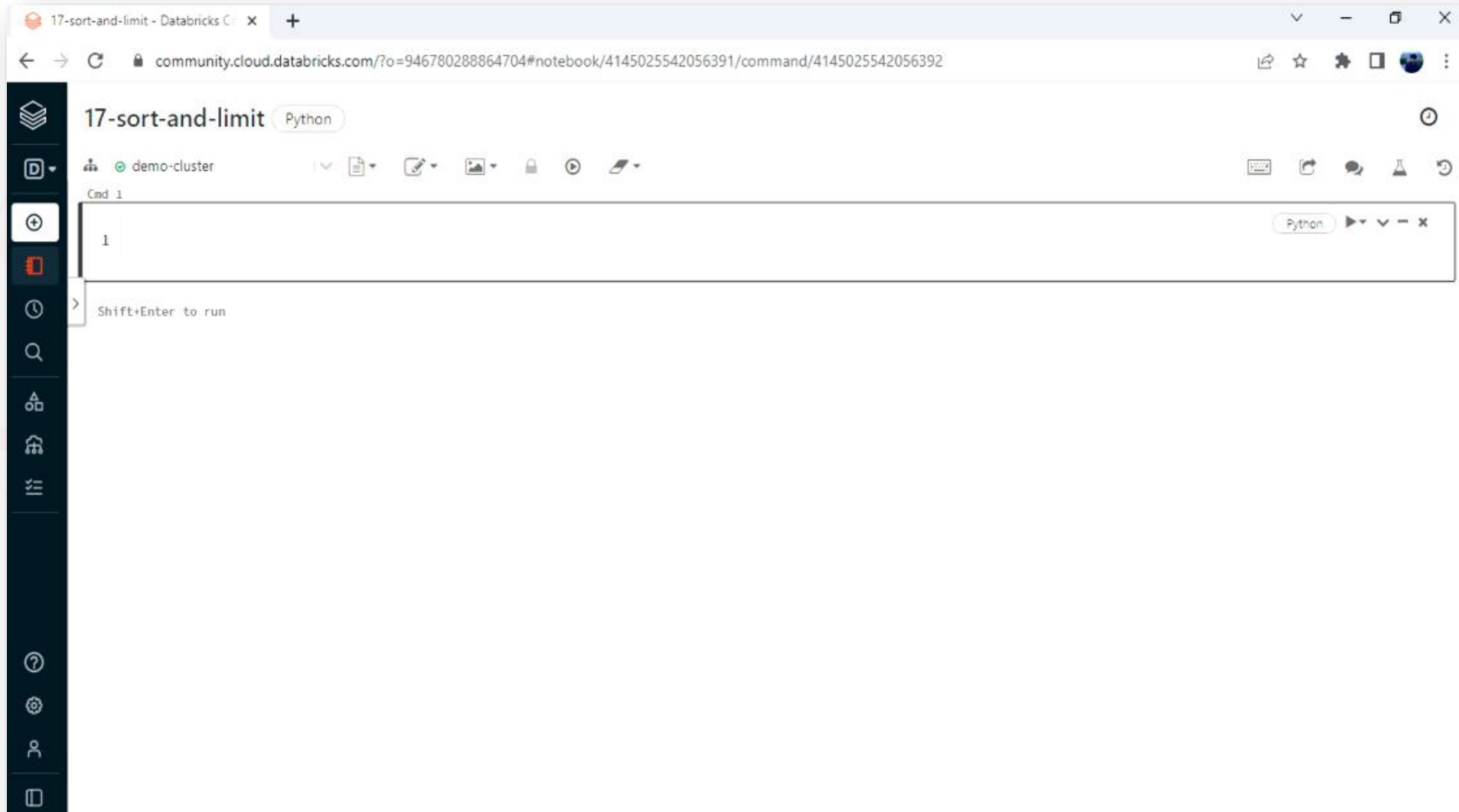
Sorting and Limiting

When we sort the Dataframe, we always want to specify two things:

1. Sorting columns
2. Sorting order

We may want to sort the Dataframe with one column or multiple columns. And sorting order could be ascending or descending. Finally, you may also have a requirement to limit the top 5 or top 10 records.

Go to your Databricks workspace and create a new notebook.
(Reference : 17-sort-and-limit)



We need a Dataframe for applying various sorting techniques. So I have created a Dataframe and displayed it as shown below.

17-sort-and-limit

Python

demo-cluster

Cmd 1

```
1 airlines_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .option("samplingRatio", "0.0001") \  
6     .load("/databricks-datasets/airlines/part-00000") \  
7 \  
8 display(airlines_df)
```

▶ (3) Spark Jobs

	Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum
1	1987	10	14	3	741	730	912	849	PS	1451	NA
2	1987	10	15	4	729	730	903	849	PS	1451	NA
3	1987	10	17	6	741	730	918	849	PS	1451	NA
4	1987	10	18	7	729	730	847	849	PS	1451	NA
5	1987	10	19	1	749	730	922	849	PS	1451	NA
6	1987	10	21	3	728	730	848	849	PS	1451	NA
7	1987	10	22	4	728	730	852	849	PS	1451	NA

Truncated results, showing first 1000 rows.

[Click to re-execute with maximum result limits.](#)

Command took 4.17 seconds -- by prashant@scholarnest.com at 5/9/2022, 7:57:00 PM on demo-cluster

Now let's assume I want to summarize the count of airline data over the following fields:

1. UniqueCarrier
2. Origin
3. Dest
4. FlightNum

In order to achieve this requirement, we will group by the Dataframe over these fields and calculate the count.

We need a Dataframe for applying various sorting techniques. So I have created a Dataframe and displayed it as shown below.

17-sort-and-limit

Python

demo-cluster

Cmd 1

```
1 airlines_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .option("samplingRatio", "0.0001") \  
6     .load("/databricks-datasets/airlines/part-00000") \  
7 \  
8 display(airlines_df)
```

▶ (3) Spark Jobs

	Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum
1	1987	10	14	3	741	730	912	849	PS	1451	NA
2	1987	10	15	4	729	730	903	849	PS	1451	NA
3	1987	10	17	6	741	730	918	849	PS	1451	NA
4	1987	10	18	7	729	730	847	849	PS	1451	NA
5	1987	10	19	1	749	730	922	849	PS	1451	NA
6	1987	10	21	3	728	730	848	849	PS	1451	NA
7	1987	10	22	4	728	730	852	849	PS	1451	NA

Truncated results, showing first 1000 rows.

[Click to re-execute with maximum result limits.](#)

Command took 4.17 seconds -- by prashant@scholarnest.com at 5/9/2022, 7:57:00 PM on demo-cluster

Here is my summary Dataframe shown below. Now let's assume you want to sort this Dataframe over the count. Spark gives you two methods for sorting your Dataframe: `sort()` and `orderBy()`. Both the methods are the same. There is no difference, and you can use any one of these. We will be using `orderBy()` in all our examples. But you can replace the `orderBy()` call with the `sort()` method, and the code should work without any problems.

Cmd 2

```
1 summary_df = airlines_df.groupBy("UniqueCarrier", "Origin", "Dest", "FlightNum").count()
2 display(summary_df)
```

(2) Spark Jobs

	UniqueCarrier	Origin	Dest	FlightNum	count
1	PS	PDX	SFO	1874	58
2	TW	CVG	DAY	22	45
3	TW	FLL	LGA	110	59
4	TW	STL	PIT	138	46
5	TW	LIT	STL	284	24
6	TW	STL	LAX	403	52
7	TW	LGA	PBI	453	56

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 7.38 seconds -- by prashant@scholarnest.com at 5/9/2022, 7:59:32 PM on demo-cluster

Here is the code to sort the Dataframe on count. And this one is a single-column count. The default order is ascending. So you will see the smallest count first, and the largest value comes in the end.

17-sort-and-limit Python

demo-cluster

Cmd 3:

```
1 summary_df.orderBy("count").show()
```

(2) Spark Jobs

UniqueCarrier	Origin	Dest	FlightNum	count
UA	CID	DSM	455	1
UA	LAX	SEA	1796	1
UA	LAX	SAN	197	1
UA	ORD	SRQ	468	1
UA	MDW	ORD	994	1
UA	ORD	CLE	846	1
UA	SEA	ANC	1796	1
UA	SAN	HNL	197	1
HP	PHX	PUB	418	1
UA	PHL	ORD	85	1
UA	CLE	ORD	1733	1
HP	GCN	PHX	1246	1
EA	BOS	ALB	835	1
UA	LGA	PHL	85	1
UA	LAX	SMF	1888	1
UA	LAS	SMF	839	1
EA	MIA	SJU	1701	1
UA	SRO	RSW	468	1

Command took 5.97 seconds -- by prashant@scholarnest.com at 5/9/2022, 8:00:38 PM on demo-cluster

Here I changed the order by column name to a SQL-like expression and added a desc clause, but if you see the output, you will observe that this does not work. So be careful with this common mistake.

17-sort-and-limit Python

demo-cluster

```
1 from pyspark.sql.functions import expr
2
3 summary_df.orderBy(expr("count desc")).show()
```

(2) Spark Jobs

UniqueCarrier	Origin	Dest	FlightNum	count
UA	CID	DSM	455	1
UA	LAX	SEA	1796	1
UA	LAX	SAN	197	1
UA	ORD	SRQ	468	1
UA	MDW	ORD	994	1
UA	ORD	CLE	846	1
UA	SEA	ANC	1796	1
UA	SAN	HNL	197	1
HP	PHX	PUB	418	1
UA	PHL	ORD	85	1
UA	CLE	ORD	1733	1
HP	GCN	PHX	1246	1
EA	BOS	ALB	835	1
UA	LGA	PHL	85	1
UA	LAX	SMF	1888	1
UA	LAS	SMF	839	1
EA	MIA	SJU	1701	1
UA	SRO	RSW	468	1

Command took 5.90 seconds -- by prashant@scholarnest.com at 5/9/2022, 8:02:28 PM on demo-cluster

The orderBy() method takes another argument for specifying the order. The second argument sets ascending to false, and the orderBy() method starts showing results in descending order. The default value of ascending is true, so you can ignore the second argument if you want to sort your result in ascending order.

17-sort-and-limit Python

demo-cluster

1 summary_df.orderBy("count", ascending=False).show()

(2) Spark Jobs

UniqueCarrier	Origin	Dest	FlightNum	count
UA	DEN	ICT	540	61
UA	MCO	ORD	427	61
UA	PWM	ORD	320	61
UA	ICT	ORD	446	61
UA	SEA	LAX	1201	61
UA	PDX	SFO	183	61
UA	MCI	ORD	946	61
TW	MSN	STL	504	61
UA	ORD	SYR	896	61
UA	EW	SFO	35	61
UA	ORD	SFO	121	61
UA	ORD	DFW	311	61
UA	TUL	DEN	251	61
UA	ORD	LNK	557	61
UA	LAX	ORD	502	61
UA	SFO	SEA	462	61
UA	SFO	ORD	128	61
TW	SAT	STL	526	61

Command took 8.06 seconds -- by prashant@scholarnest.com at 5/9/2022, 8:04:16 PM on demo-cluster

So you can import the desc() function and apply it to the column. This is another way to sort the order in descending order.

17-sort-and-limit Python

demo-cluster

Cmd 6

```
1 from pyspark.sql.functions import desc
2
3 summary_df.orderBy(desc("count")).show()
```

(2) Spark Jobs

UniqueCarrier	Origin	Dest	FlightNum	count
UA	DEN	ICT	540	61
UA	MCO	ORD	427	61
UA	PWM	ORD	320	61
UA	ICT	ORD	446	61
UA	SEA	LAX	1201	61
UA	PDX	SFO	183	61
UA	MCI	ORD	946	61
TW	MSN	STL	504	61
UA	ORD	SYR	896	61
UA	EWB	SFO	35	61
UA	ORD	SFO	121	61
UA	ORD	DFW	311	61
UA	TUL	DEN	251	61
UA	ORD	LNK	557	61
UA	LAX	ORD	502	61
UA	SFO	SEA	462	61
UA	SFO	ORD	128	61
TW	SAT	STL	526	61

We have another alternative to sort the columns in descending order. We can import the col() method and apply it to the column name to convert it into a column object. Once we have a column object, we can apply column APIs.

17-sort-and-limit Python

demo-cluster Cmd 7

```
1 from pyspark.sql.functions import col
2
3 summary_df.orderBy(col("count").desc()).show()
```

(2) Spark Jobs

UniqueCarrier	Origin	Dest	FlightNum	count
UA	DEN	ICT	540	61
UA	MCO	ORD	427	61
UA	PWM	ORD	320	61
UA	ICT	ORD	446	61
UA	SEA	LAX	1201	61
UA	PDX	SFO	183	61
UA	MCI	ORD	946	61
TW	MSN	STL	504	61
UA	ORD	SYR	896	61
UA	EWR	SFO	35	61
UA	ORD	SFO	121	61
UA	ORD	DFW	311	61
UA	TUL	DEN	251	61
UA	ORD	LNK	557	61
UA	LAX	ORD	502	61
UA	SFO	SEA	462	61
UA	SFO	ORD	128	61
TWI	SAT	STL	526	61

Now let us look how to sort multiple columns.

I want to sort the Dataframe by carrier code, origin, destination, and count. By default, all the columns are sorted in ascending order. Inside the same carrier code, it is sorted by the source. Similarly, it is then ordered by the destination code and finally by the count. I didn't sort it by flight number, so the flight number appears random.

```
Cmd 8

1 summary_df.orderBy("UniqueCarrier", "Origin", "Dest", "count").show()

> (2) Spark Jobs

+-----+-----+-----+-----+-----+
|UniqueCarrier|Origin|Dest|FlightNum|count|
+-----+-----+-----+-----+-----+
|AA|ABQ|DFW|459|28|
|AA|ABQ|DFW|188|30|
|AA|ABQ|DFW|922|30|
|AA|ABQ|DFW|232|31|
|AA|ABQ|DFW|314|31|
|AA|ABQ|DFW|216|31|
|AA|ABQ|ELP|859|31|
|AA|ABQ|ORD|522|30|
|AA|ABQ|ORD|434|31|
|AA|ALB|ORD|639|28|
|AA|ALB|ORD|617|29|
|AA|ALB|ORD|641|31|
|AA|ALB|ORD|379|31|
|AA|AMA|DFW|948|31|
|AA|AMA|DFW|472|31|
|AA|AMA|DFW|880|31|
|AA|AMA|LBB|494|31|
|AA|ANC|SEA|2728|27|

Command took 5.42 seconds -- by prashant@scholarnest.com at 5/9/2022, 8:14:07 PM on demo-cluster
```

I want to keep the first three fields sorted in ascending order, but the count must be in descending order. And you can see the code and output for this requirement shown below.

17-sort-and-limit

Python

demo-cluster

Cmd 8

1

from pyspark.sql.functions import desc

2

3

summary_df.orderBy("UniqueCarrier", "Origin", "Dest", desc("count")).show()

(2) Spark Jobs

UniqueCarrier	Origin	Dest	FlightNum	count
AA	ABQ	DFW	314	31
AA	ABQ	DFW	216	31
AA	ABQ	DFW	232	31
AA	ABQ	DFW	188	30
AA	ABQ	DFW	922	30
AA	ABQ	DFW	459	28
AA	ABQ	ELP	859	31
AA	ABQ	ORD	434	31
AA	ABQ	ORD	522	30
AA	ALB	ORD	379	31
AA	ALB	ORD	641	31
AA	ALB	ORD	617	29
AA	ALB	ORD	639	28
AA	AMA	DFW	948	31
AA	AMA	DFW	472	31
AA	AMA	DFW	880	31
AA	AMA	LBB	494	31
AA	ANC	SEA	2706	28

Spark also offers you another method to do it as shown below. We pass a list of sorting columns. Then we pass a list of Boolean values. In this example, the ascending flag for the first three columns is true, and the last one is false. So the orderBy() will sort the Dataframe in ascending order for the first three columns, and the last one is descending.

17-sort-and-limit

Python

demo-cluster

```
1 summary_df.orderBy(["UniqueCarrier", "Origin", "Dest", "count"], ascending=[1,1,1,0]).show()
```

(2) Spark Jobs

UniqueCarrier	Origin	Dest	FlightNum	count
AA	ABQ	DFW	314	31
AA	ABQ	DFW	216	31
AA	ABQ	DFW	232	31
AA	ABQ	DFW	188	30
AA	ABQ	DFW	922	30
AA	ABQ	DFW	459	28
AA	ABQ	ELP	859	31
AA	ABQ	ORD	434	31
AA	ABQ	ORD	522	30
AA	ALB	ORD	379	31
AA	ALB	ORD	641	31
AA	ALB	ORD	617	29
AA	ALB	ORD	639	28
AA	AMA	DFW	948	31
AA	AMA	DFW	472	31
AA	AMA	DFW	880	31
AA	AMA	LBB	494	31
AA	ANC	SEA	2706	28

Command took 5.96 seconds -- by prashant@scholarnest.com at 5/9/2022, 8:18:28 PM on demo-cluster

In some cases, you may want to see only top N records. Spark allows you to do that using the `limit()` method.

You can see in the screenshot below that I can limit this Dataframe to return only the top-five results. So we got only the top five records. Limit is beneficial with single-column sorting requirements.

```
> Cmd 10

1  from pyspark.sql.functions import desc
2
3  summary_df.orderBy("UniqueCarrier", "Origin", "Dest", desc("count")).limit(5).show()

▶ (2) Spark Jobs

+-----+-----+-----+-----+-----+
|UniqueCarrier|Origin|Dest|FlightNum|count|
+-----+-----+-----+-----+
|          AA|   ABQ|  DFW|      314|   31|
|          AA|   ABQ|  DFW|      216|   31|
|          AA|   ABQ|  DFW|      232|   31|
|          AA|   ABQ|  DFW|      188|   30|
|          AA|   ABQ|  DFW|      922|   30|
+-----+-----+-----+-----+

Command took 5.82 seconds -- by prashant@scholarnest.com at 5/9/2022, 8:20:26 PM on demo-cluster
```

Here is another example.

I am grouping the Dataframe records by FlightNum and taking sum(distance). Then I sorted it in descending order to the largest travelled distance comes at the top. Finally, I am limiting the Dataframe to five records. The result of the set of transformations gave me the top five flights that travelled most.

17-sort-and-limit Python

demo-cluster

Cmd 11

```
1 from pyspark.sql.functions import sum, desc
2
3 airlines_df.groupBy("FlightNum") \
4     .agg(sum("Distance").alias("total_distance")) \
5     .sort(desc("total_distance")) \
6     .limit(5) \
7     .show()
```

▶ (2) Spark Jobs

FlightNum	total_distance
1	1211179
2	1151736
85	948446
91	850182
8	834451

Command took 5.16 seconds -- by prashant@scholarnest.com at 5/9/2022, 8:23:22 PM on demo-cluster



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com